

Programación funcional

Lambdas, métodos de referencia y streams

4.1 ¿Qué es la programación funcional?

Si le pides a un compañero que te avise cuando lleguen pedidos de más de 50€, no le explicas paso a paso cómo mirar la lista: le das una regla ("que supere los 50€") y él la aplica.

Eso es lo que vas a hacer aquí: en vez de escribir tú el bucle, le das a Java la regla.

Es un paradigma que se apoya en dos ideas: los datos no cambian de valor una vez creados (inmutabilidad) y las funciones se tratan como datos, encadenando la salida de una como entrada de la siguiente.

Cuando filtras o transformas una lista con estas herramientas, la lista original no se toca: se genera un resultado nuevo, y personas sigue teniendo los mismos elementos que antes.

4.1 Imperativo vs funcional

Programación imperativa vs programación funcional

Dos formas de resolver el mismo problema: obtener la lista de personas mayores de edad.

Programación imperativa "Cómo hacerlo"



Paso a paso

- Indicas **explícitamente** los pasos que debe seguir el programa.
- Utilizas **bucles**, **condiciones** y **variables** intermedias.
- Sueles **modificar** el estado de las variables.
- Controlas el **recorrido** y el momento de cada operación.

Ejemplo en Java

Con bucle y condición

```
1 List<Persona> adultos = new ArrayList<>();
2 for (int i = 0; i < personas.size(); i++) {
3     Persona p = personas.get(i);
4     if (p.getEdad() >= 18) {
5         adultos.add(p);
6     }
7 }
```

Cómo se ejecuta

Recorrer
la lista



Comprobar
la condición



Añadir
si cumple



Más código y más detalles

Funciona perfectamente, pero requiere escribir y mantener todos los pasos del proceso.

Programación funcional "Qué quiero obtener"



Reglas y composición

- Describes **qué resultado** quieres, no cómo recorrer la lista.
- Utilizas **funciones** (lambdas) y operaciones de alto nivel.
- Trabajas con datos **inmutables** (la colección original no se modifica).
- Puedes encadenar operaciones (composición de funciones).

Ejemplo en Java

Con streams y lambda

```
1 List<Persona> adultos = personas.stream()
2     .filter(p -> p.getEdad() >= 18)
3     .toList();
4
```

Cómo se ejecuta

Lista de
personas



filter

(p -> p.getEdad() >= 18)



Nueva lista
con adultos



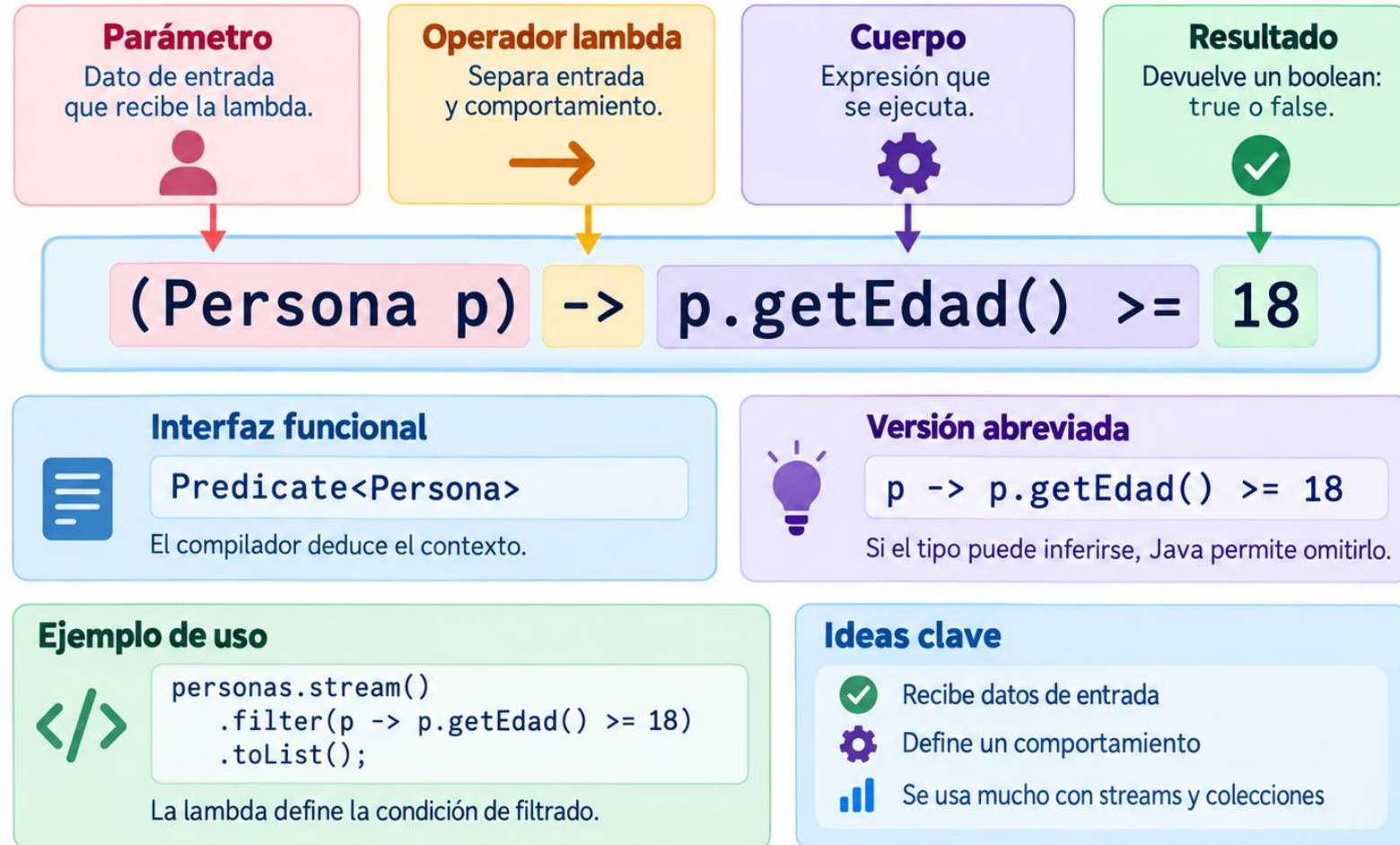
Más compacto y expresivo

Dejas que el lenguaje se encargue de recorrer los datos. El resultado es más legible y mantenible.

4.2 Funciones lambda — sintaxis

Anatomía de una función lambda en Java

Una lambda es una forma compacta de expresar el comportamiento de una interfaz funcional.



4.2 Cuidado: una lambda no tiene tipo fijo

Java necesita saber qué interfaz funcional estás implementando (cuántos parámetros, de qué tipo, qué debe devolver) para darle sentido a la lambda. Esa información la saca del contexto: el tipo de la variable, o el tipo que espera el método al que se la pasas.

```
Comparator<Persona> comp = (p1, p2) -> p2.getEdad() - p1.getEdad();
```

Por eso una lambda nunca aparece "sola": siempre va asignada a algo o pasada como argumento a algo.

4.2 Ejemplo: ordenar personas por edad

Antes de las lambdas, ordenar con un criterio propio exigía implementar Comparator en una clase aparte:

Antes de las lambdas

```
class ComparadorPersona
    implements Comparator<Persona> {
    @Override
    public int compare(
        Persona p1,
        Persona p2) {
        return p2.getEdad()
            - p1.getEdad();
    }
}

personas.sort(
    new ComparadorPersona());
```

Con una lambda

```
personas.sort((p1, p2) ->
    p2.getEdad() - p1.getEdad());
```

4.3 Métodos de referencia

Un método de referencia es una forma abreviada de escribir una lambda que se limita a llamar a un método que ya existe, con la sintaxis `Clase::metodo`.

Tipo	Ejemplo	Equivale a
Método estático	<code>String::toUpperCase</code>	<code>s -> s.toUpperCase()</code>
Método de instancia	<code>Persona::getNombre</code>	<code>p -> p.getNombre()</code>
Método de un objeto arbitrario	<code>Integer::sum</code>	<code>(a, b) -> a + b</code>

4.3 Ejemplo de métodos de referencia

Los tres tipos de la tabla, usados dentro de un stream real:

```
List<String> nombres = List.of("ANA", "LUIS", "MARÍA");

// Estático: String::toUpperCase
nombres.stream().map(String::toUpperCase)
        .forEach(System.out::println); // ANA, LUIS, MARÍA

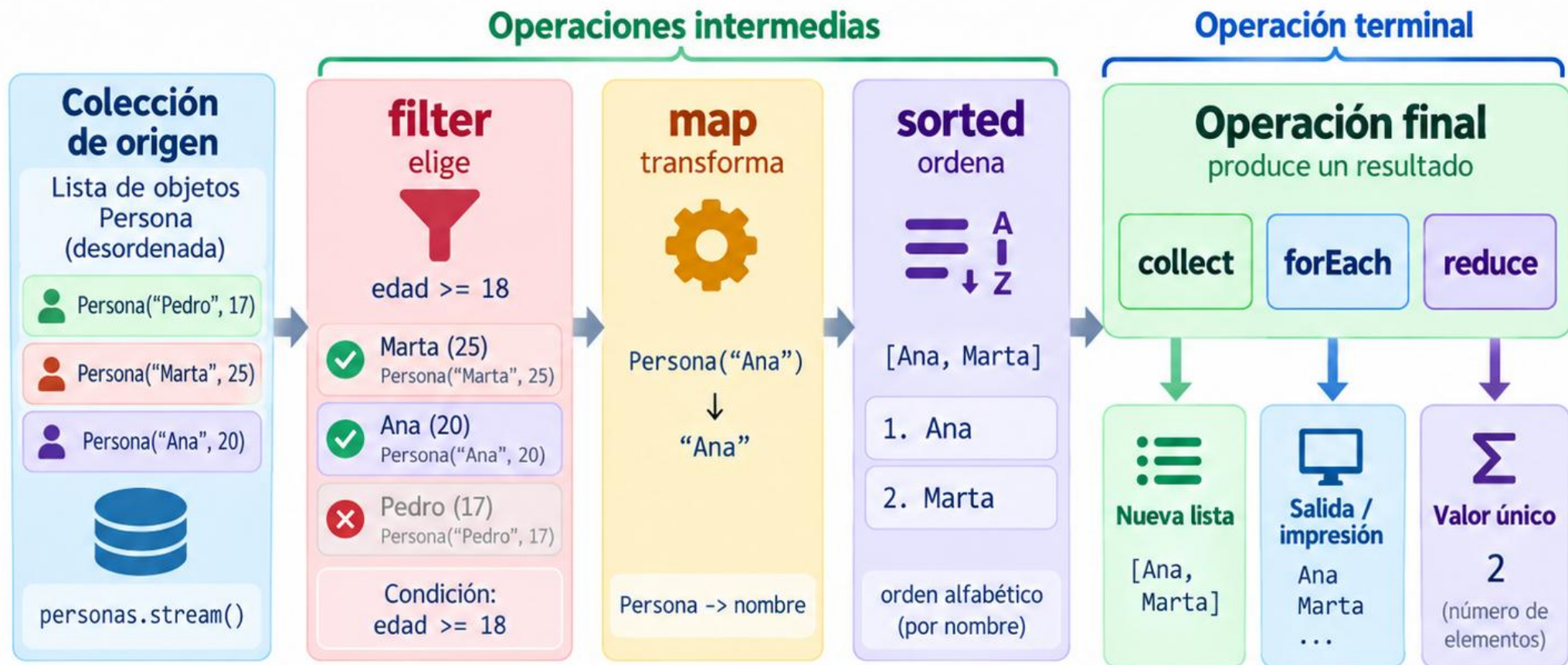
// De instancia: Persona::getNombre
List<String> soloNombres = personas.stream()
        .map(Persona::getNombre).toList();

// De un objeto arbitrario: Integer::sum
int total = List.of(10, 20, 30).stream()
        .reduce(0, Integer::sum); // 60
```

4.4 Streams — la cadena de montaje

Cómo funciona un stream en Java

Una forma declarativa y funcional de procesar colecciones



4.4 filter — filtrar por un criterio

Se queda solo con los elementos que cumplen la condición indicada, igual que haría un if dentro de un bucle, pero sin bucle.

Imperativo

```
List<Persona> adultos =  
    new ArrayList<>();  
for (Persona p : personas) {  
    if (p.getEdad() >= 18) {  
        adultos.add(p);  
    }  
}
```

Funcional

```
Stream<Persona> adultos =  
    personas.stream()  
        .filter(p ->  
            p.getEdad() >= 18);
```

4.4 map — transformar cada elemento

Aplica una función a cada elemento y devuelve un stream con los resultados, no con las personas originales.

Imperativo

```
List<Integer> edades =  
    new ArrayList<>();  
for (Persona p : personas) {  
    if (p.getEdad() >= 18) {  
        edades.add(p.getEdad());  
    }  
}
```

Funcional

```
Stream<Integer> edades =  
    personas.stream()  
        .filter(p ->  
            p.getEdad() >= 18)  
        .map(Persona::getEdad);
```

4.4 sorted — ordenar los elementos

Ordena el stream según el Comparator que le pases, igual que hacía sort en el apartado anterior.

Imperativo

```
List<Persona> adultos =
    new ArrayList<>();
for (Persona p : personas) {
    if (p.getEdad() >= 18)
        adultos.add(p);
}
Collections.sort(adultos,
    (p1, p2) ->
        p2.getEdad() - p1.getEdad());
```

Funcional

```
Stream<Persona> ordenadas =
    personas.stream()
        .filter(p ->
            p.getEdad() >= 18)
        .sorted((p1, p2) ->
            p2.getEdad() - p1.getEdad());
```

4.4 collect y forEach — operaciones finales

toList() → convierte el resultado en una lista

```
List<Integer> edades = personas.stream()
    .filter(p -> p.getEdad() >= 18)
    .map(Persona::getEdad)
    .toList(); // atajo de .collect(Collectors.toList())
```

collect() → une el resultado en un texto

```
String texto = personas.stream()
    .map(Persona::getEdad)
    .collect(Collectors.joining(", ", "Adultos: ", ""));
```

forEach() → recorre el resultado para hacer algo con él

```
personas.stream()
    .filter(p -> p.getEdad() >= 18)
    .forEach(System.out::println);
```

4.4 reduce — reducir a un solo valor

Combina todos los elementos en uno solo usando una función de acumulación. Es el equivalente funcional de un acumulador que actualizas dentro de un bucle:

Imperativo

```
List<Integer> edades =  
    Arrays.asList(20, 30, 40);  
  
int suma = 0;  
for (int edad : edades) {  
    suma += edad;  
}
```

Funcional

```
List<Integer> edades =  
    Arrays.asList(20, 30, 40);  
  
int suma = edades.stream()  
    .reduce(0, Integer::sum); // 90
```

El valor inicial debe ser el que no cambia el resultado al combinarlo: 0 para sumar, 1 si multiplicaras. Además sirve de resultado por defecto si el stream estuviera vacío. Por eso existe una versión sin valor inicial, que devuelve un Optional en vez de un int.

4.4 average — la media de un stream

```
double mediaEdadAdultos = personas.stream()  
    .filter(p -> p.getEdad() >= 18)  
    .mapToInt(Persona::getEdad)  
    .average()  
    .getAsDouble();
```

`average()` no devuelve un `double`, sino un `OptionalDouble`: si el stream estuviera vacío no habría media que calcular, y `getAsDouble()` lanzaría una excepción.

No hace falta memorizar todas las operaciones de golpe: con `filter`, `map`, `sorted`, `collect` y `forEach` resuelves la mayoría de los casos.

Cómo encajan todas las piezas

Cómo encajan todas las piezas

Programación funcional en Java

Funciones.
Colecciones.
Resultados reales.

Mismas
piezas.
Grandes
posibilidades.



Lambda

Función sin nombre escrita en el momento.

```
p -> p.getEdad() >= 18
```



Interfaz funcional

Define la forma de la función.

```
Predicate<T>  
Function<T, R>  
Comparator<T>  
Consumer<T>
```

La lambda tiene una forma determinada por una...

La lambda también puede escribirse como un...



Método de referencia

Atajo cuando ya existe un método.

```
Persona::getNombre
```

Lambdas o métodos de referencia se usan en las operaciones de un...



Stream

Cadena de operaciones sobre una colección.

```
filter map sorted ...
```



Ejemplo integrado: del dato al resultado

Las piezas trabajan juntas

Código más expresivo.
Programas más claros.



Colección de entrada

List<Persona>

```
List.of(  
    new Persona("Luis", 17),  
    new Persona("Ana", 25),  
    new Persona("Marta", 30),  
    new Persona("Pablo", 20)  
)
```

```
personas.stream()
```

Convierte la colección en un stream.



Stream<Persona>



```
filter(...)
```

Espera un Predicate<Persona>

```
p -> p.getEdad() >= 18
```

Filtra solo las personas mayores de edad.



```
sorted(...)
```

Espera un Comparator<Persona>

```
(a, b) ->  
    a.getNombre().  
    compareTo(b.getNombre())
```

Ordena alfabéticamente por nombre.



```
map(...)
```

Espera un Function<Persona, String>

```
Persona::getNombre
```

Extrae al nombre de cada persona.



```
toList()
```

Operación terminal.

Recopila los resultados en una lista.



Resultado final

```
[Ana, Marta, Pablo]
```

¡Solo personas mayores de edad!



Idea clave:

los streams procesan colecciones usando funciones.

Esas funciones suelen escribirse como lambdas o métodos de referencia, y Java sabe qué forma deben tener gracias a las interfaces funcionales.

Programación funcional
para un código más simple,
potente y legible.